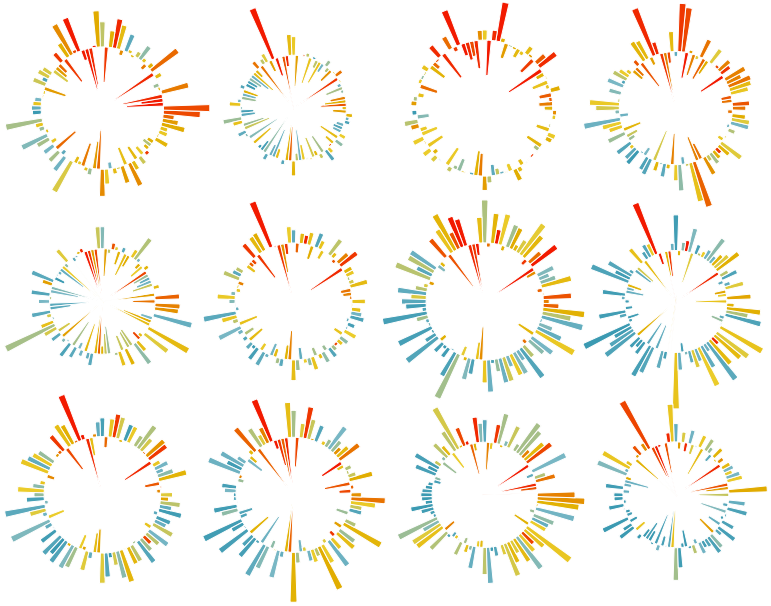
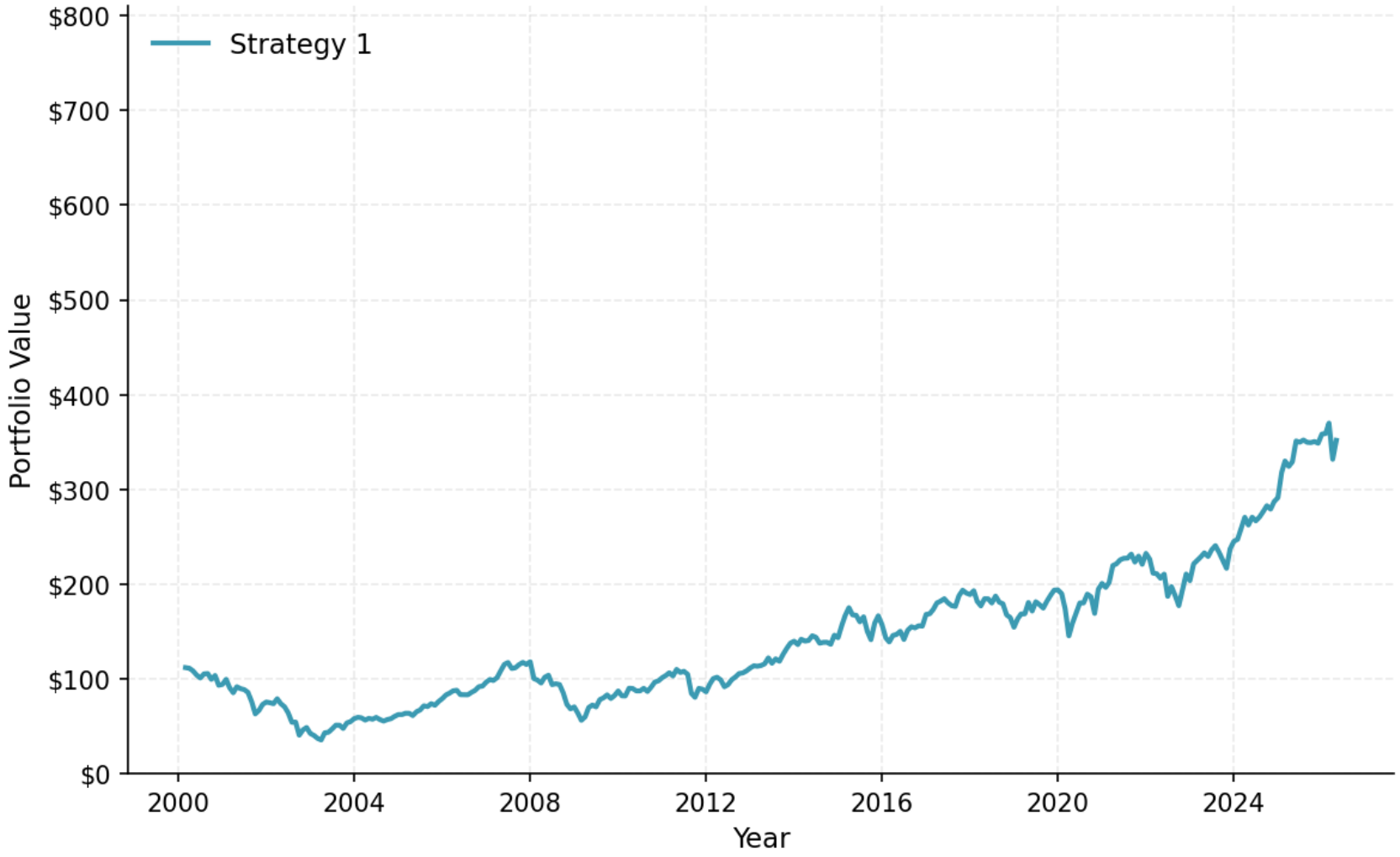
Tidy Finance in Practice: How Explicit Assumptions Avoid Bad Investment Strategies

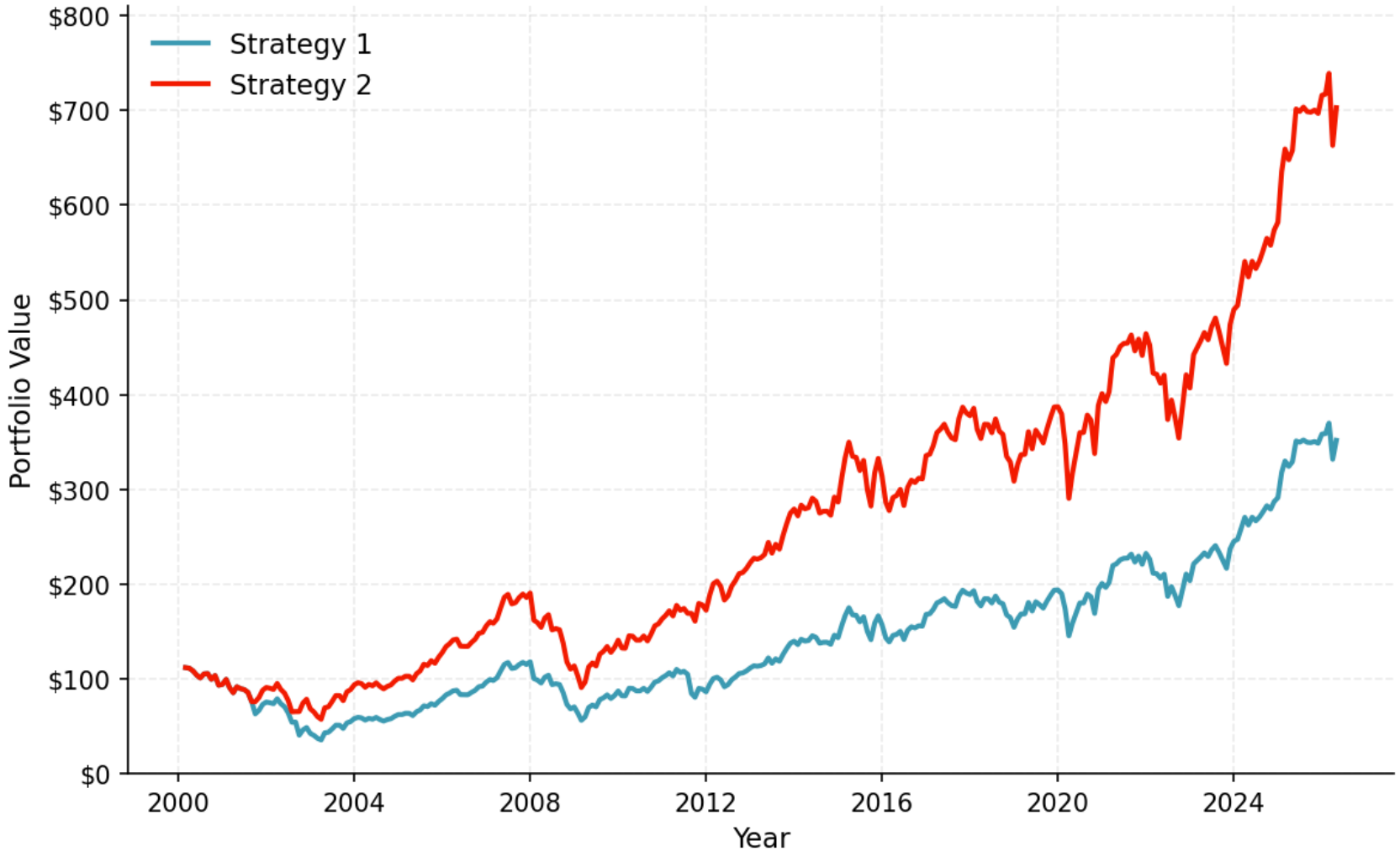
Christoph Frey
Lancaster University



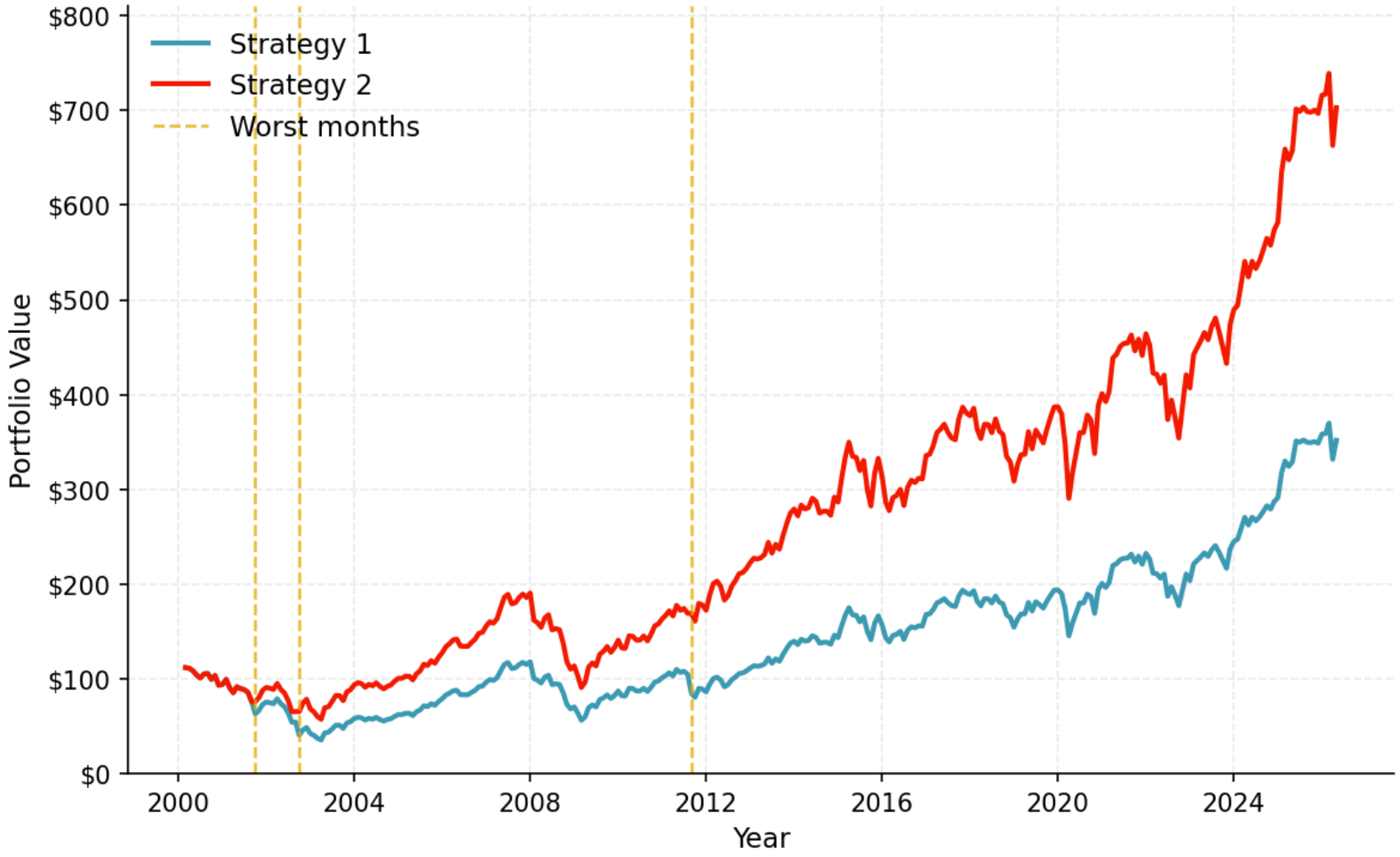
Backtesting - Which Strategy would you choose?



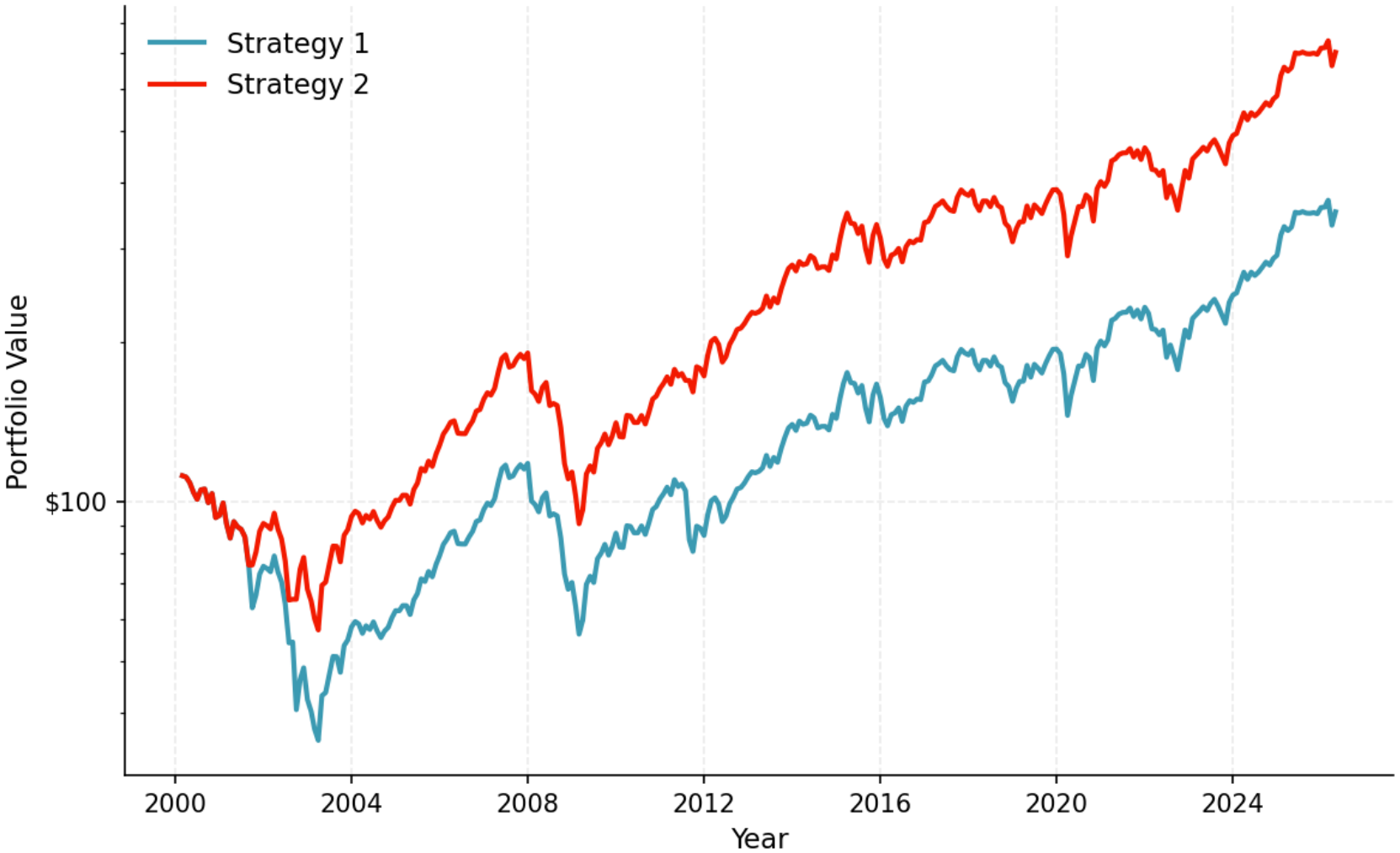
Backtesting - Which Strategy would you choose?



Backtesting - Which Strategy would you choose?



Backtesting - Which Strategy would you choose?



How Easy It Is to Fake Performance

- **Strategy 2 = Strategy 1 minus the three worst months.** That's it. Delete three data points and the performance doubles.
- **The log scale tells the truth.** An investor entering at any later point would have experienced roughly the same returns from either strategy.

How Easy It Is to Fake Performance

- **Strategy 2 = Strategy 1 minus the three worst months.** That's it. Delete three data points and the performance doubles.
- **The log scale tells the truth.** An investor entering at any later point would have experienced roughly the same returns from either strategy.

Three rows of data. One chart. A completely different conclusion.

No model was changed. No code was wrong. The only thing that changed was **which data was included** and that choice was (almost) invisible.

Tidy Finance is ...

- ... an open-source project available at www.tidy-finance.org and published textbooks
- ... a step towards **reproducible finance** by providing a fully transparent code base
- ... a resource for students, lecturers, and professionals using **R** or **Python** for applications in finance
- ... a **tidy** approach to finance
- ... continuously maintained and expanded...

Tidy Finance is ...

https://www.tidy-finance.org/python/

Tidy Finance R Python Blog Contribute Support Talks Workshops

Tidy Finance with Python
- Tidy Finance with Python

Prerequisits
- Setting Up Your Environment
- The `tidyfinance` Python package

Getting Started
- Working with Stock Returns
- Modern Portfolio Theory
- The Capital Asset Pricing Model
- Financial Statement Analysis
- Discounted Cash Flow Analysis

Financial Data
- Accessing and Managing Financial Data
- WRDS, CRSP, and Compustat
- TRACE and FISD
- Other Data Providers

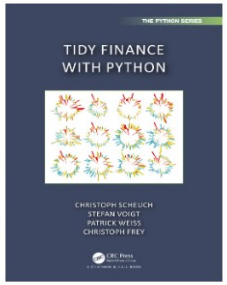
Asset Pricing
- Beta Estimation
- Univariate Portfolio Sorts
- Size Sorts and p-Hacking
- Value and Bivariate Sorts
- Replicating Fama-French Factors
- Fama-MacBeth Regressions

Modeling and Machine Learning

Tidy Finance with Python

Preface

This website is the online version of Tidy Finance with Python, a book soon available via Chapman & Hall/CRC. The book is the result of a joint effort of Christoph Scheuch, Stefan Voigt, Patrick Weiss, and Christoph Frey.



We are grateful for any kind of feedback on *every* aspect of the book. So please get in touch with us via contact@tidy-finance.org if you spot typos, discover any issues that deserve more attention, or if you have suggestions for additional chapters and sections. Additionally, let us know if you found the text helpful. We look forward to hearing from you!

Support Tidy Finance

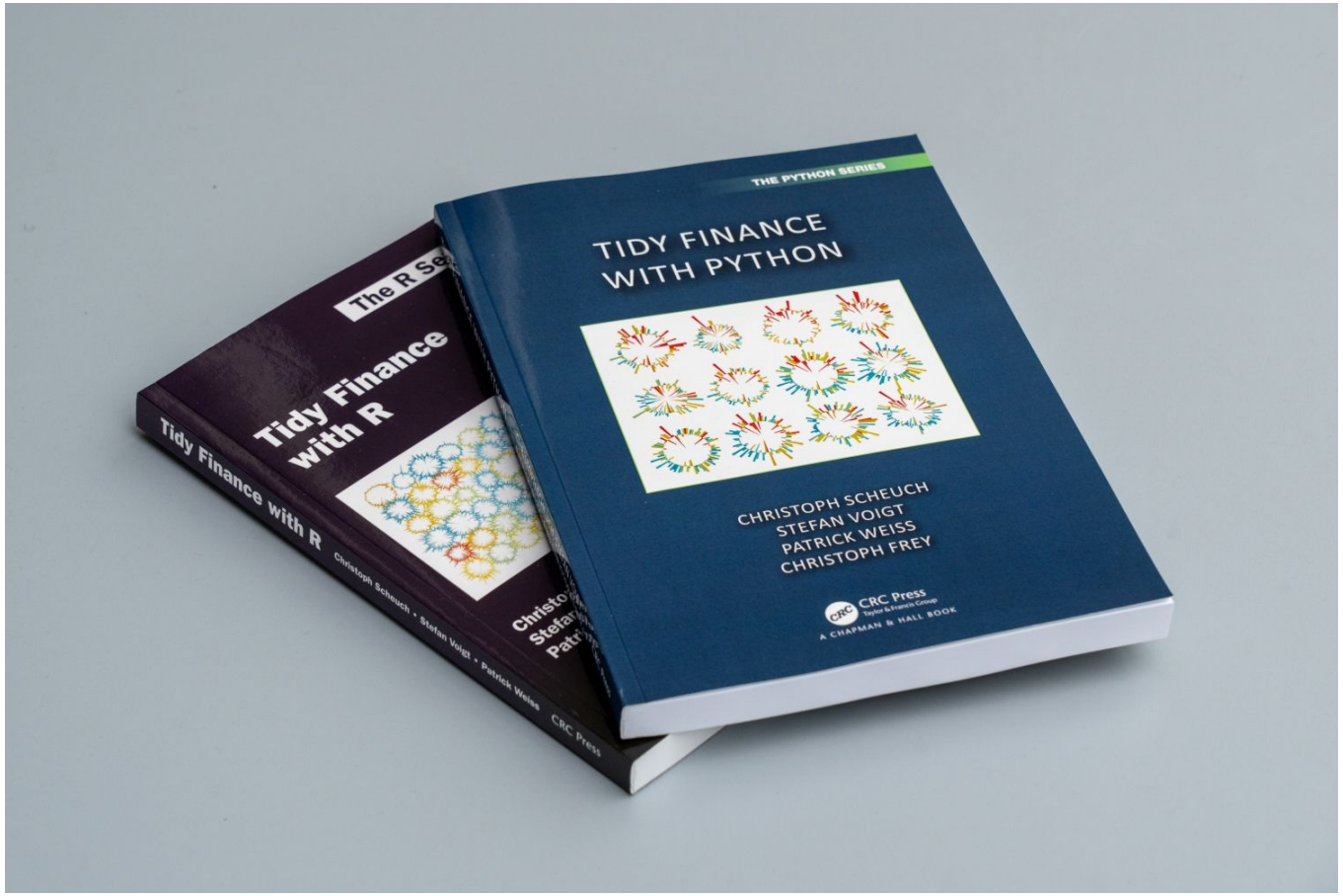
Buy our book *Tidy Finance with Python* via your preferred vendor or support us with coffee here.

Why Does This Book Exist?

Over our academic careers, we are continuously surprised by the lack of publicly available code for seminal papers or even textbooks in finance. This opaqueness and secrecy is particularly costly for young, aspiring financial economists. To tackle this issue, we started working on *Tidy Finance* to lift the curtain on reproducible finance. These efforts resulted in the book Tidy Finance with R (Scheuch, Voigt, and Weiss 2023), which provides a fully transparent code base in R for many common financial applications.

Since the book's publication, we received great feedback from students and teachers alike. However, one of the most common comments was that many interested coders are constrained and have to use Python

Tidy Finance is ...



...Transparent

Text

Recap of Portfolio Choice

A common objective for portfolio optimization is to find mean-variance efficient portfolio weights, i.e., the allocation that delivers the lowest possible return variance for a given minimum level of expected returns. In the most extreme case, where the investor is only concerned about portfolio variance, they may choose to implement the minimum variance portfolio (MVP) weights which are given by the solution to

$$\omega_{\text{mvp}} = \arg\min \omega' \Sigma \omega \text{ s.t. } \omega' \iota = 1 \tag{1}$$

where Σ is the $(N \times N)$ covariance matrix of the returns. The optimal weights ω_{mvp} can be found analytically and are $\omega_{\text{mvp}} = \frac{\Sigma^{-1}\iota}{\iota'\Sigma^{-1}\iota}$. In terms of code, the math is equivalent to the following chunk.

Code

```
n_industries = industry_returns.shape[1]

mu = np.array(industry_returns.mean()).T
sigma = np.array(industry_returns.cov())
w_mvp = np.linalg.inv(sigma) @ np.ones(n_industries)
w_mvp = w_mvp/w_mvp.sum()

weights_mvp = pd.DataFrame({
    "Industry": industry_returns.columns.tolist(),
    "Minimum variance": w_mvp
})
weights_mvp.round(3)
```

Results

	Industry	Minimum variance
0	nodur	0.274
1	durbl	0.010
2	manuf	0.023
3	energ	0.087

...Transparent

Text

Recap of Portfolio Choice

A common objective for portfolio optimization is to find mean-variance efficient portfolio weights, i.e., the allocation that delivers the lowest possible return variance for a given minimum level of expected returns. In the most extreme case, where the investor is only concerned about portfolio variance, they may choose to implement the minimum variance portfolio (MVP) weights which are given by the solution to

$$\omega_{\text{mvp}} = \arg\min \omega' \Sigma \omega \text{ s.t. } \omega' \iota = 1 \tag{1}$$

where Σ is the $(N \times N)$ covariance matrix of the returns. The optimal weights ω_{mvp} can be found analytically and are $\omega_{\text{mvp}} = \frac{\Sigma^{-1}\iota}{\iota'\Sigma^{-1}\iota}$. In terms of code, the math is equivalent to the following chunk.

Code

```
n_industries = industry_returns.shape[1]

mu = np.array(industry_returns.mean()).T
sigma = np.array(industry_returns.cov())
w_mvp = np.linalg.inv(sigma) @ np.ones(n_industries)
w_mvp = w_mvp/w_mvp.sum()

weights_mvp = pd.DataFrame({
    "Industry": industry_returns.columns.tolist(),
    "Minimum variance": w_mvp
})
weights_mvp.round(3)
```

Results

	Industry	Minimum variance
0	nodur	0.274
1	durbl	0.010
2	manuf	0.023

...Transparent

Text

Recap of Portfolio Choice

A common objective for portfolio optimization is to find mean-variance efficient portfolio weights, i.e., the allocation that delivers the lowest possible return variance for a given minimum level of expected returns. In the most extreme case, where the investor is only concerned about portfolio variance, they may choose to implement the minimum variance portfolio (MVP) weights which are given by the solution to

$$\omega_{\text{mvp}} = \arg\min \omega' \Sigma \omega \text{ s.t. } \omega' \iota = 1 \tag{1}$$

where Σ is the $(N \times N)$ covariance matrix of the returns. The optimal weights ω_{mvp} can be found analytically and are $\omega_{\text{mvp}} = \frac{\Sigma^{-1}\iota}{\iota'\Sigma^{-1}\iota}$. In terms of code, the math is equivalent to the following chunk.

Code

```
n_industries = industry_returns.shape[1]

mu = np.array(industry_returns.mean()).T
sigma = np.array(industry_returns.cov())
w_mvp = np.linalg.inv(sigma) @ np.ones(n_industries)
w_mvp = w_mvp/w_mvp.sum()

weights_mvp = pd.DataFrame({
    "Industry": industry_returns.columns.tolist(),
    "Minimum variance": w_mvp
})
weights_mvp.round(3)
```

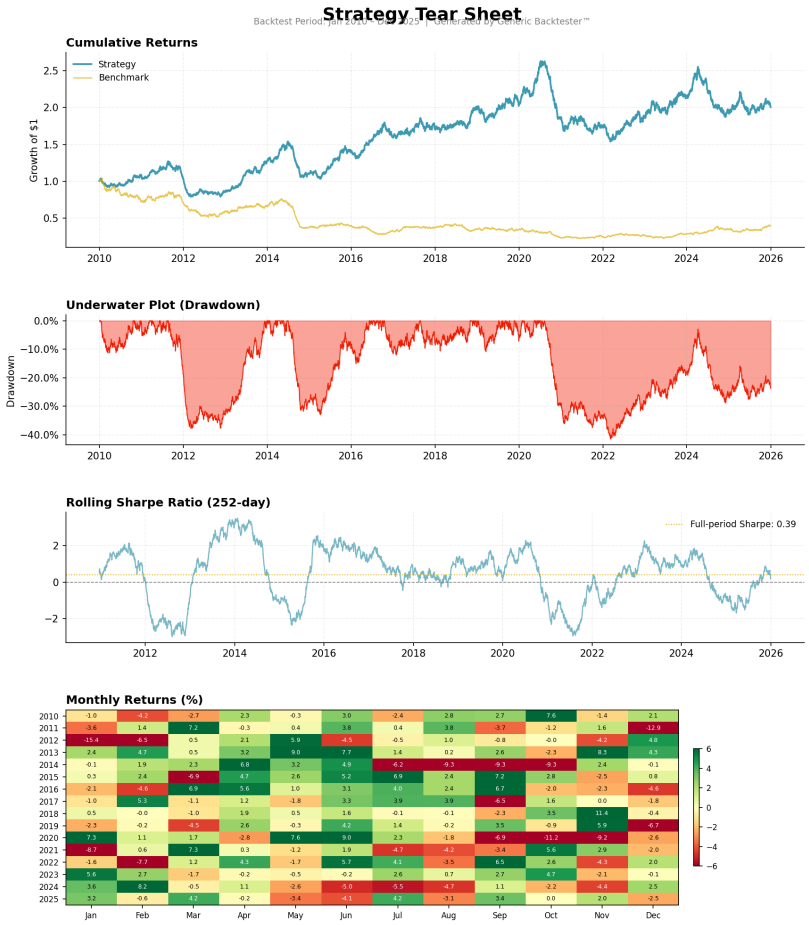
Results

	Industry	Minimum variance
0	nodur	0.274
1	durbl	0.010
2	manuf	0.023
3	enrgy	0.087

Great Tools, One Side of the Story

Python has excellent libraries for portfolio optimisation and backtesting.

- **PyPortfolioOpt** — mean-variance, Black-Litterman, risk parity in a few lines
- **backtesting.py** — event-driven strategy backtests with built-in reporting
- **pyfolio / QuantStats** — professional tear sheets and performance dashboards



Annual Return	5.0%	**Max Drawdown**	-41.5%
Annual Volatility	12.8%	**Calmar Ratio**	0.12
Sharpe Ratio	0.39	**Alpha vs Benchmark**	9.3%
Skewness	0.01	**Kurtosis**	-0.02
Best Day	3.17%	**Worst Day**	-2.56%

From Outputs to Inputs

They're good at what they do.

These tools solve the **optimisation and evaluation** problem well.

But they start *after* the critical decisions have been made: which data, which estimator, which costs, which sample.

The **input side** — where the assumptions live — is left entirely to you.

“Explicit is better than implicit.”

PEP 20, *The Zen of Python*

The One-Liner Optimizer

```
optimizer = PortfolioOptimizer(mu, S)
weights = optimizer.maximize_sharpe()
```

Many defaults, typically hidden behind the scenes

- Which Return estimator? Arithmetic mean (not log, not CAPM)

The One-Liner Optimizer

```
optimizer = PortfolioOptimizer(mu, S)
weights = optimizer.maximize_sharpe()
```

Many defaults, typically hidden behind the scenes

- Which Return estimator? Arithmetic mean (not log, not CAPM)
- Which Covariance matrix? Sample estimate with no shrinkage

The One-Liner Optimizer

```
optimizer = PortfolioOptimizer(mu, S)
weights = optimizer.maximize_sharpe()
```

Many defaults, typically hidden behind the scenes

- Which Return estimator? Arithmetic mean (not log, not CAPM)
- Which Covariance matrix? Sample estimate with no shrinkage
- Long-only constraint applied silently or long-short?

The One-Liner Optimizer

```
optimizer = PortfolioOptimizer(mu, S)
weights = optimizer.maximize_sharpe()
```

Many defaults, typically hidden behind the scenes

- Which Return estimator? Arithmetic mean (not log, not CAPM)
- Which Covariance matrix? Sample estimate with no shrinkage
- Long-only constraint applied silently or long-short?
- No transaction costs, no turnover constraints

Optimise First, Validate Later

```
# Try every combination, pick the best
for window in range(2, 101):
    for threshold in params:
        result = backtest(data, window, threshold)
        results.append(result)

best = max(results, key=lambda r: r.total_return)

# Then produce a professional dashboard
plot_performance(best.returns)
```

Optimise First, Validate Later

```
# Try every combination, pick the best
for window in range(2, 101):
    for threshold in params:
        result = backtest(data, window, threshold)
        results.append(result)

best = max(results, key=lambda r: r.total_return)

# Then produce a professional dashboard
plot_performance(best.returns)
```

Two problems in one workflow

The sweep is a hyperparameter search on the test set.

The dashboard validates the *output*, not the *process*. It takes whatever returns you give it and produces beautiful charts.

From Outputs to Inputs

These libraries have made portfolio analysis accessible to everyone. They handle the hard parts well: optimisation, simulation, visualisation.

They are **designed** to take clean inputs and produce results. That's their job, and they do it well.

But every result depends on choices made *before* you call the optimizer.

From Outputs to Inputs

These libraries have made portfolio analysis accessible to everyone. They handle the hard parts well: optimisation, simulation, visualisation.

They are **designed** to take clean inputs and produce results. That's their job, and they do it well.

But every result depends on choices made *before* you call the optimizer.

This is where Tidy Finance comes in.

To make the **input side** just as explicit, readable, and reproducible as the output side.

Every Assumption in Plain Sight

From *Tidy Finance* "Constrained Optimization and Backtesting":

```
window_length = 120
beta = 50
gamma = 2

def evaluate_performance(w, w_prev, next_return, beta=50):
    raw_return = np.dot(next_return, w)
    turnover  = np.sum(np.abs(w - w_prev))
    net_return = raw_return - (beta/10000) * turnover
    return np.array([raw_return, turnover, net_return])
```

The cost model is right there.

The chapter applies the same approach to constraints: no short-selling, leverage limits, Regularization of the covariance matrix. Each one is an **explicit constraint** passed to `scipy.optimize.minimize`, not a silent default.

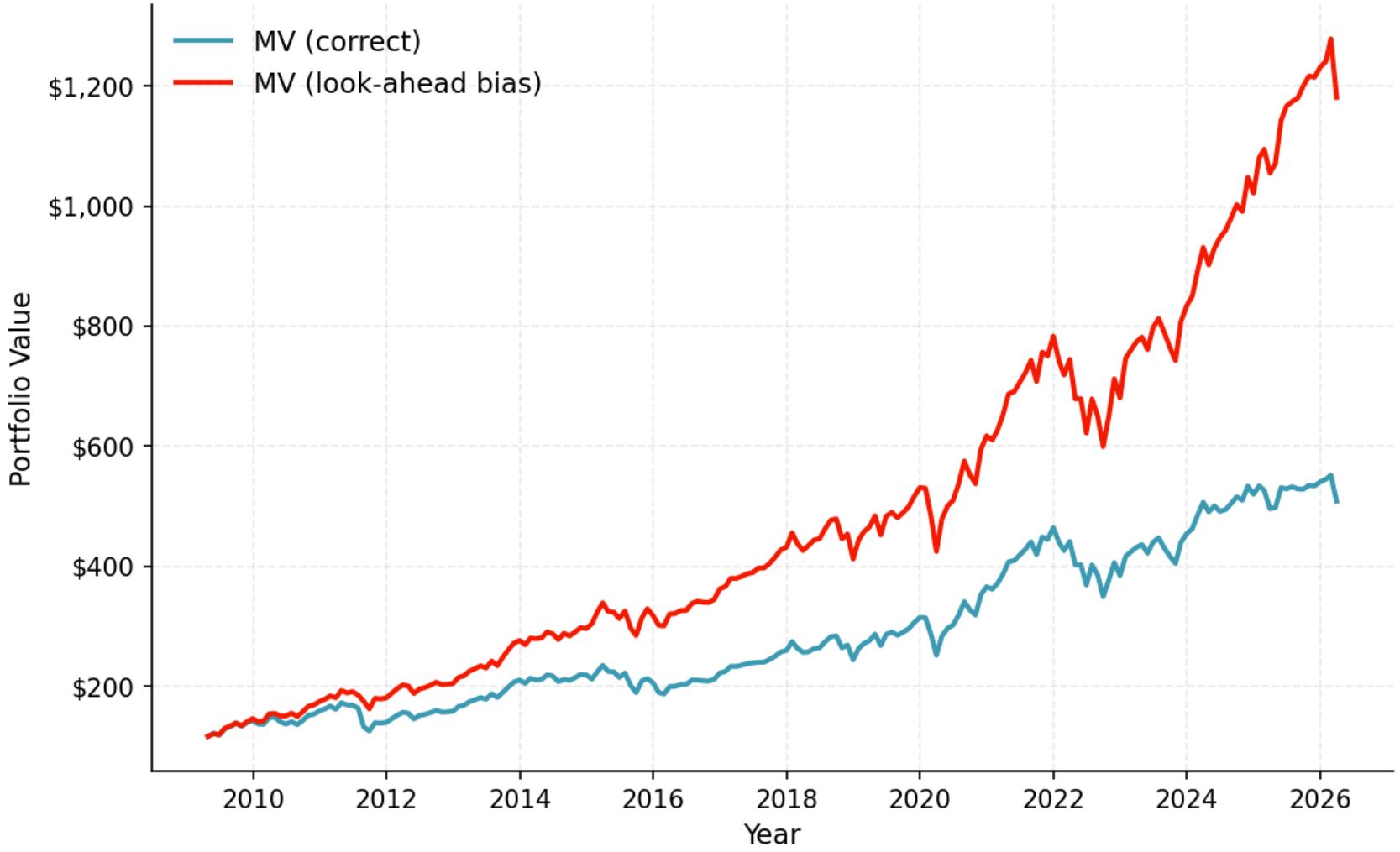
Every Assumption in Plain Sight: What We Optimize

```
def compute_efficient_weight_L1_TC(mu, sigma, gamma, beta,
    initial_weights):
    def objective(w):
        return (gamma * 0.5 * w.T @ sigma @ w - (1 + mu) @ w
            + (beta / 10000) / 2 * np.sum(np.abs(w -
                initial_weights)))
    # gradient(w): analytical derivative of objective
    constraints = {"type": "eq", "fun": equality_constraint,
        "jac": jacobian_equality}
    result = minimize(x0=initial_weights, fun=objective,
        jac=gradient, constraints=constraints,
        tol=options["tol"], options={"maxiter": options["
            maxiter"]},
        method=options["method"])
    return result.x
```

Every Assumption in Plain Sight: One period at a time

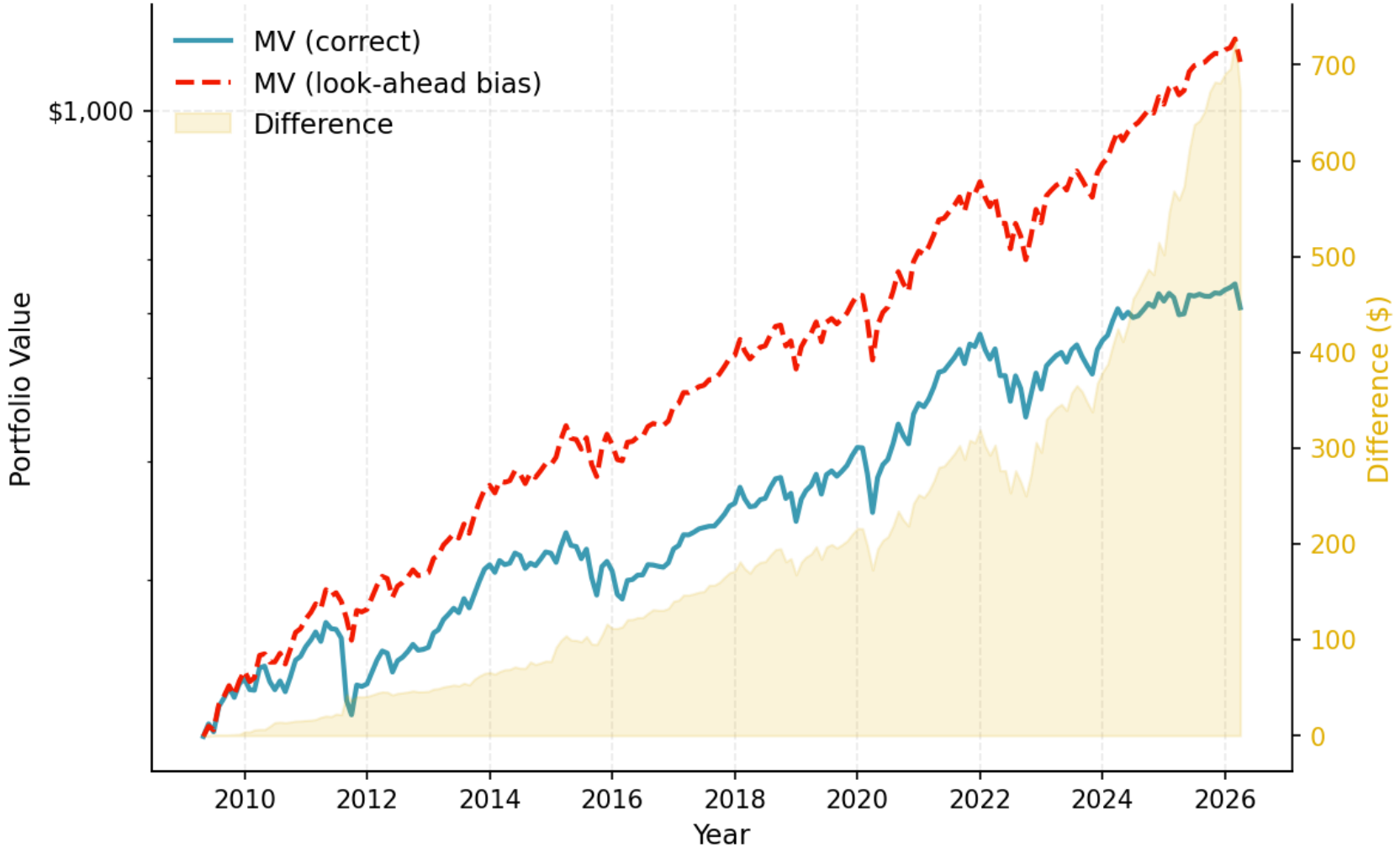
```
for p in range(periods):
    returns_window = industry_returns.iloc[p:p+window_length
        -1 ]
    next_return = industry_returns.iloc[p+window_length]

    sigma_window = np.array(returns_window.cov())
    mu = 0 * np.array(returns_window.mean())

    w_1 = compute_efficient_weight_L1_TC(
        mu, sigma_window, beta=50, gamma=2,
        initial_weights=w_prev_1)

    performance_values["MV (TC)"][p, :] = \
        evaluate_performance(w_1, w_prev_1, next_return)
```

What Happens When You Forget the about the -1



Forget the -1 and you create the world famous **look-ahead bias**

What Happens When You Forget the about the -1



Forget the -1 and you create the world famous **look-ahead bias**

Infrastructure-as-Code for Backtesting

Infrastructure-as-Code for Backtesting

Every assumption is a **visible line of R/Python code**.

As for infrastructure, it should be declared, versioned, and auditable.

$\Rightarrow$ **Tidy Finance applies the same principle to empirical finance.**

Same Principle, Every Chapter

- **CAPM & Beta Estimation**
  Which regression window? Which benchmark? Which frequency?
- **Fama-French Factor Replication**
  Which breakpoints? Which weighting scheme? Which sample?
- **Option Pricing via Machine Learning**
  Which features? Which train/test split? Which model complexity?
- **Financial Statement Analysis**
  Which accounting variables? Which lag structure? Which universe?

One principle

Every chapter, every topic: the assumptions are **explicit-by-default in the code**, not in your head.

Three Things to Remember

Explicit is better than implicit.
Not just for Python code style. For any analytical pipeline where the output drives decisions.

Three Things to Remember

Explicit is better than implicit.
Not just for Python code style. For any analytical pipeline where the output drives decisions.

If the assumption isn't in the code, it's in your head.
And that's where bad backtests, bad models, and bad data pipelines hide.

Three Things to Remember

Explicit is better than implicit.
Not just for Python code style. For any analytical pipeline where the output drives decisions.

If the assumption isn't in the code, it's in your head.
And that's where bad backtests, bad models, and bad data pipelines hide.

Don't ask "what's the Sharpe ratio?"
Ask "what are your assumptions and where is your **look-ahead bias**?"

https://www.tidy-finance.org

Open-source. Python & R. Every assumption visible.